



**POLITECHNIKA
BYDGOSKA**

Wydział Telekomunikacji,
Informatyki i Elektrotechniki

Zaawansowane programowanie obiektowe - laboratorium

Laboratorium nr: 2

**Tytuł: IMPLEMENTACJA WALIDATORA DANYCH Z
WYKORZYSTANIEM ADNOTACJI, REFLEKSJI I
WZORCA STRATEGIA**

Szymon Banaszek, 1 grupa

Nr legitymacji: 124983

Prowadzący dr inż. Damian Szczegielniak

Bydgoszcz, dnia: 09.04.2026

Ćwiczenie

Ćwiczenie realizuje **walidację obiektów** (np. klasy [Student](#)) na podstawie adnotacji (np. [@NotNull](#), [@Email](#), [@Size](#)). Pozwala to automatycznie sprawdzać poprawność danych wprowadzanych do obiektu.

Najważniejsze klasy i mechanizmy

1. Klasa [Student](#)

Przykładowy model danych z adnotacjami walidacyjnymi:

```
1. public class Student {
2.     @NotNull(message = "Nie podano imienia")
3.     @Size(min = 3, max = 50, message = "Imię musi składać się z przynajmniej {min} znaków i nie
przekraczać {max}")
4.     private String imie;
5.
6.     @NotNull(message = "Nie podano nazwiska")
7.     @Size(min = 2, max = 50, message = "Nazwisko musi składać się z przynajmniej {min} znaków i
nie przekraczać {max}")
8.     private String nazwisko;
9.
10.    @NotNull(message = "Nie podano numeru indeksu")
11.    @NrIndeksu(message = "Niepoprawny numer indeksu")
12.    private String nrIndeksu;
13.
14.    @NotEmpty(message = "Nie podano adresu e-mail")
15.    @Email(message = "Niepoprawny adres e-mail")
16.    private String email;
17.    // ...getter/setter...
18. }
19.
```

2. Klasa [Validator](#)

Centralny mechanizm walidacji:

```
1. public class Validator {
2.     public static void validate(Object object) throws ValidationException {
3.         final List<String> errors = new ArrayList<>();
4.         final Class<?> clazz = object.getClass();
5.         for (Field field : clazz.getDeclaredFields()) {
6.             field.setAccessible(true);
7.             try {
8.                 Object value = field.get(object);
9.                 for (Annotation annotation : field.getAnnotations()) {
10.                     ValidationStrategy strategy = ValidationStrategyFactory.getStrategy(annotation);
```

```

11.         if (strategy != null) {
12.             Optional<String> validationError = strategy.validate(field, value);
13.             if(validationError.isPresent()) errors.add(validationError.get());
14.         }
15.     }
16.     } catch (IllegalAccessException e) {
17.         errors.add("Błąd dostępu do pola: " + field.getName());
18.     }
19. }
20. if (!errors.isEmpty()) throw new ValidationException(String.join("\n", errors));
21. }
22. }
23.

```

- Przechodzi po wszystkich polach obiektu, sprawdza adnotacje i wywołuje odpowiednią strategię walidacji.

3. Fabryka strategii walidacji

```

1. public class ValidationStrategyFactory {
2.     private static final Map<Class<? extends Annotation>, ValidationStrategy> strategies = new
HashMap<>();
3.     static {
4.         strategies.put(NotNull.class, new NotNullStrategy());
5.         strategies.put(NotEmpty.class, new NotEmptyStrategy());
6.         strategies.put(Email.class, new EmailStrategy());
7.         strategies.put(NrIndeksu.class, new NrIndeksuStrategy());
8.         strategies.put(Size.class, new SizeStrategy());
9.     }
10.    public static ValidationStrategy getStrategy(Annotation annotation) {
11.        return strategies.get(annotation.annotationType());
12.    }
13. }

```

- Mapuje typ adnotacji na odpowiednią strategię walidacji.

4. Przykładowe strategie walidacji

- **NotNullStrategy:**

```

1. public class NotNullStrategy implements ValidationStrategy {
2.     public Optional<String> validate(Field field, Object value) {
3.         if (field.isAnnotationPresent(NotNull.class) && value == null) {
4.             NotNull annotation = field.getAnnotation(NotNull.class);
5.             String errorInfo = String.format("Pole %s: %s", field.getName(), annotation.message());
6.             return Optional.of(errorInfo);
7.         }
8.         return Optional.empty();

```

```
9.     }
10. }
```

- **EmailStrategy:**

```
1. public class EmailStrategy implements ValidationStrategy {
2.     private static final Pattern EMAIL_PATTERN = Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$");
3.     public Optional<String> validate(Field field, Object value) {
4.         if (field.isAnnotationPresent(Email.class) && value != null
5.             && (!String.class.equals(value.getClass()) || !EMAIL_PATTERN.matcher((String)
value).matches())) {
6.             Email annotation = field.getAnnotation(Email.class);
7.             String errorInfo = String.format("Pole %s: %s", field.getName(), annotation.message());
8.             return Optional.of(errorInfo);
9.         }
10.        return Optional.empty();
11.    }
12. }
13. }
```

- **SizeStrategy:**

```
1. public class SizeStrategy implements ValidationStrategy {
2.     public Optional<String> validate(Field field, Object value) {
3.         if (field.isAnnotationPresent(Size.class) && value != null &&
(String.class.equals(value.getClass()))) {
4.             Size annotation = field.getAnnotation(Size.class);
5.             int length = ((String) value).length();
6.             if (length < annotation.min() || length > annotation.max()) {
7.                 String message = annotation.message()
8.                     .replace("{min}", String.valueOf(annotation.min()))
9.                     .replace("{max}", String.valueOf(annotation.max()));
10.                String errorInfo = String.format("Pole %s: %s", field.getName(), message);
11.                return Optional.of(errorInfo);
12.            }
13.        }
14.        return Optional.empty();
15.    }
16. }
17. }
```

5. Przykład użycia (Main)

```
1. public class Main {
2.     public static void main(String[] args) {
3.         try {
4.             Student student = new Student();
5.             student.setImie("Grzegorz");
6.             student.setNazwisko("Bręczyszczykiewicz");
7.             student.setNrIndeksu("12345678");
8.             student.setEmail("Grzegorz.Bręczyszczykiewicz@pbs.edu.pl");
9.             Validator.validate(student);
10.        } catch (ValidationException e) {
11.            System.out.println(e.getMessage());
12.        }
13.    }
14. }
15.
```

Podsumowanie działania

1. Tworzony jest obiekt (np. [Student](#)) z danymi.
2. Pola klasy są oznaczone adnotacjami walidacyjnymi.
3. Metoda [Validator.validate\(\)](#) sprawdza każde pole, wywołując odpowiednią strategię walidacji.
4. W przypadku błędów rzucany jest wyjątek z opisem problemów.

Wnioski.

- Poznałem zastosowanie adnotacji do walidacji danych w Javie.
- Zastosowanie wzorca strategii umożliwia łatwe dodawanie nowych reguł walidacji.
- Ćwiczenie pokazuje, jak oddzielić logikę walidacji od modelu danych.